Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 1er. Semestre 2026

Proyecto 1 - Simulación de Enfermedades Epidemiológicas

202400554 – Raúl José Daniel Paredes González

Resumen

Este ensayo documenta el desarrollo de un simulador de propagación de enfermedades implementado en C# con Programación Orientada a Objetos. El sistema analiza rejillas de tejido celular aplicando reglas similares al Juego de la Vida para clasificar enfermedades en leves, graves o mortales. Se construyeron listas enlazadas manuales para gestionar pacientes y estados de simulación, y se utilizó Graphviz para visualizar cada período. La aplicación carga archivos XML, ejecuta simulaciones automáticas o paso a paso, detecta patrones repetidos y genera reportes XML con diagnósticos. El proyecto evidencia cómo la combinación de POO, estructuras de datos lineales y herramientas de visualización ofrece una solución funcional para problemas de simulación epidemiológica.

Palabras clave

1. Simulación
2. Autómatas celulares
3. Enfermedades
4. Listas enlazadas
5. POO

Abstract

This essay documents the development of a disease propagation simulator implemented in C# using Object-Oriented Programming. The system analyzes cell tissue grids, applying rules similar to the Game of Life to classify diseases as mild, severe, or fatal. Manual linked lists were constructed to manage patients and simulation states, and Graphviz was used to visualize each period. The application loads XML files, runs automatic or step-by-step simulations, detects recurring patterns, and generates XML reports with diagnoses. The project demonstrates how the combination of OOP, linear data structures, and visualization tools offers a functional solution for epidemiological simulation problems.

Keywords

1. Simulation

2. Cellular Automata

3. Diseases

4. Linked Lists

5. Object-Oriented Programming

Introducción

La simulación de fenómenos biológicos mediante computadora se ha convertido en una herramienta fundamental para la investigación científica. En particular, el estudio de cómo se propagan las enfermedades a nivel celular permite anticipar comportamientos y consecuencias sin necesidad de experimentación real. El proyecto surge de la necesidad del laboratorio de investigación epidemiológica de Guatemala por contar con una herramienta que, a partir de muestras de tejido representadas en rejillas, pueda predecir si una enfermedad será leve, grave o mortal. ¿Cómo determinar si un patrón de contagio se repetirá? ¿En qué momento una enfermedad se vuelve incurable? Estas preguntas guían el desarrollo de la solución. A lo largo del documento se exploran los fundamentos teóricos de los autómatas celulares, la implementación de listas enlazadas manuales y el uso de Graphviz para visualización, demostrando cómo estas tecnologías convergen para resolver un problema real.

1. Autómatas Celulares y Simulación Epidemiológica

Los autómatas celulares son modelos matemáticos que permiten simular sistemas complejos a partir de reglas simples. El Juego de la Vida, creado por John Conway en 1970, es el ejemplo más conocido: una rejilla bidimensional donde cada celda cambia su estado según el número de vecinos vivos. En este proyecto, se adaptaron estas reglas al contexto epidemiológico: las celdas representan células del tejido humano que pueden estar sanas (0) o contagiadas (1). Las reglas de transición determinan cómo evoluciona la enfermedad período tras período, permitiendo observar patrones de propagación.
En el caso de inclusión de figuras, deben ser nítidas, legibles en blanco y negro. Se denomina figuras a gráficas, esquemas, fotografías u otros elementos gráficos.

2. Programación Orientada a Objetos en C#

La Programación Orientada a Objetos (POO) constituyó la base arquitectónica del sistema. Este paradigma permite organizar el código en torno a entidades del mundo real, facilitando su mantenimiento y escalabilidad. En palabras de Joyanes (2020), "la POO proporciona mecanismos de abstracción que permiten modelar problemas complejos mediante la identificación de clases y objetos".

En el simulador desarrollado, se implementaron las siguientes clases principales:

- **Celda**: Representa una unidad básica del tejido, con atributos de fila, columna y estado.
- **Rejilla**: Gestiona el conjunto de celdas y aplica las reglas de transición.
- **Paciente**: Almacena datos personales y su rejilla asociada.
- **ResultadoSimulacion**: Contiene el diagnóstico final (leve, grave o mortal) y los períodos relevantes.

3. Listas Simples Enlazadas como Estructura de Almacenamiento

Una de las restricciones fundamentales del proyecto fue la prohibición de utilizar listas genéricas de C# como List<T>. En su lugar, se implementaron manualmente listas simples enlazadas para gestionar pacientes, estados de simulación y celdas contagiadas.

La lista enlazada es una estructura de datos lineal donde cada elemento (nodo) contiene información y un puntero al siguiente nodo. Según Cairó (2018), "esta estructura es particularmente útil cuando no se conoce de antemano la cantidad de elementos a almacenar, ya que crece dinámicamente según sea necesario".

En el sistema se crearon tres listas personalizadas:

- ListaPaciente: Almacena todos los pacientes cargados desde el XML.
- ListaEstado: Guarda el historial de rejillas durante la simulación para detectar patrones repetidos.
- ListaCelda: Representa las coordenadas de las células contagiadas en una rejilla.

La implementación manual de estas listas implicó crear clases Nodo con atributos de dato y referencia al siguiente elemento, así como métodos para inserción, búsqueda y recorrido.

4. Procesamiento de Archivos XML

Los archivos XML constituyeron el medio de entrada y salida de información. El lenguaje de marcado extensible (XML) permite estructurar datos de manera jerárquica, facilitando su procesamiento automatizado (Pérez, 2020).

El archivo de entrada contiene la estructura siguiente:

```xml
<?xml version="1.0" encoding="UTF-8"?>
<pacientes>
   <paciente>
      <datospersonales>
         <nombre>Lucas</nombre>
         <edad>33</edad>
      </datospersonales>
      <periodos>15</periodos>
      <m>10</m>
      <rejilla>
         <celda f="1" c="1" />
         <celda f="2" c="1" />
         <celda f="3" c="1" />
         <celda f="4" c="1" />
         <celda f="5" c="1" />
         <celda f="6" c="1" />
         <celda f="7" c="1" />
         <celda f="8" c="1" />
         <celda f="9" c="1" />
         <celda f="10" c="1" />
         <celda f="1" c="10" />
         <celda f="2" c="10" />
         <celda f="3" c="10" />
         <celda f="4" c="10" />
         <celda f="5" c="10" />
         <celda f="6" c="10" />
         <celda f="7" c="10" />
         <celda f="8" c="10" />
         <celda f="9" c="10" />
         <celda f="10" c="10" />
      </rejilla>
   </paciente>

</pacientes>
```

Figura 1. Entrada XML.

Fuente: Universidad San Carlos de Guatemala Facultad de Ingeniería Escuela de Ciencias y Sistema.

El sistema parsea este documento utilizando las clases XmlDocument y XmlNode de .NET, extrayendo la información para construir objetos Paciente y sus rejillas asociadas. Las celdas contagiadas se almacenan en una ListaCelda, asumiendo que las no mencionadas están sanas.

5. Visualización con Graphviz

Graphviz es una herramienta de código abierto para la representación gráfica de estructuras de datos mediante el lenguaje DOT (Takeyas, 2012). En este proyecto, se utilizó para generar imágenes de las rejillas en cada período de simulación.

El proceso consiste en:

1. Crear un archivo .dot con una tabla HTML que representa la rejilla.
2. Las celdas contagiadas se colorean en rojo.
3. Ejecutar el comando dot -Tpng para convertir el archivo a formato PNG.

Esta representación visual permite a los científicos observar directamente cómo evoluciona el patrón de contagio, facilitando la identificación de comportamientos periódicos. La Figura 1 muestra un ejemplo de rejilla generada.

6. Clasificación de Enfermedades

Una vez detectado el patrón, el sistema clasifica la enfermedad según los criterios del enunciado:

- **Leve**: No se encontraron patrones repetidos en el límite de períodos.
- **Grave**: Se repite un patrón con N1 > 1 (patrón distinto al inicial) o N > 1 (patrón inicial).

- **Mortal**: Se repite un patrón con N = 1 (patrón inicial) o N1 = 1 (otro patrón).

Conclusiones

1. La implementación de autómatas celulares mediante Programación Orientada a Objetos demostró ser una estrategia efectiva para modelar problemas epidemiológicos. El sistema desarrollado logra simular la propagación de enfermedades en tejidos celulares aplicando reglas simples, lo que evidencia que fenómenos biológicos complejos pueden ser abordados computacionalmente sin requerir modelos matemáticos avanzados. Esta aproximación resulta particularmente valiosa en contextos de recursos limitados, donde la experimentación real no siempre es viable.

2. La construcción manual de listas simples enlazadas, aunque más laboriosa que utilizar estructuras predefinidas, permitió comprender a profundidad el funcionamiento de los tipos de datos abstractos y su importancia en la gestión eficiente de memoria. Esta experiencia refuerza la necesidad de que los programadores conozcan los fundamentos de las estructuras de datos, más allá de limitarse a usar las que proporcionan los lenguajes.

3. La detección de patrones repetidos constituyó el desafío técnico más significativo del proyecto. El algoritmo implementado para verificar la consistencia de las repeticiones, particularmente en casos donde N1=1, requirió múltiples iteraciones y correcciones. Esto pone de manifiesto que, en programación, la lógica subyacente a menudo es más compleja de lo que parece a simple vista, y que la depuración constituye una fase fundamental del desarrollo.

Referencias bibliográficas

Cairó, O. (2018). *Estructuras de datos*. McGraw Hill Interamericana.

Joyanes,L. (2020). *Programación orientada a objetos con C#*. McGraw-Hill.

Pérez, C. (18 de diciembre de 2020). Qué es y cómo abrir un archivo XML. *MuyInteresante*. https://www.muyinteresante.es/tecnologia/23571.html

Sirakoulis, G. C., & Bandini, S. (2012). *Cellular automata: 9th international conference on cellular automata for research and industry*. Springer.

Takeyas,B. L. (2012). *Estructura de datos orientados a objetos: Pseudocódigo y aplicaciones en C#.NET*. Alfa Omega.

Anexos

1. Diagrama de Clases

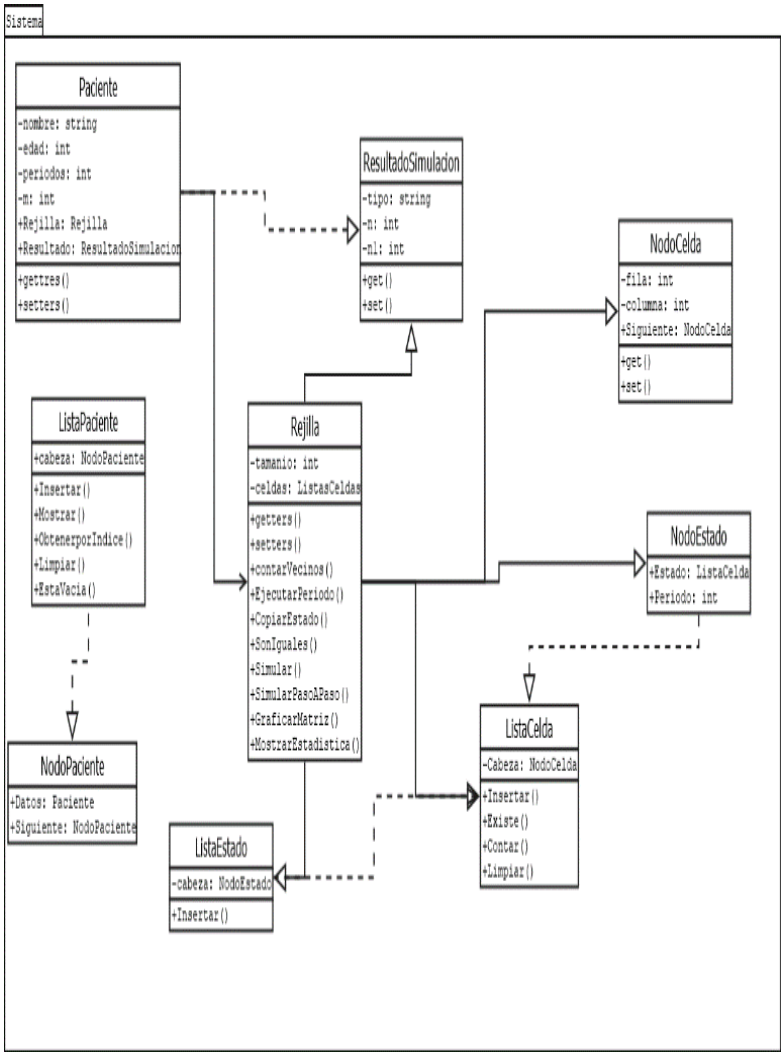

Figura 2. Diagrama de clases.

Fuente: Imagen propia.

2. Diagrama de Actividades

- **Aplicación de Reglas**

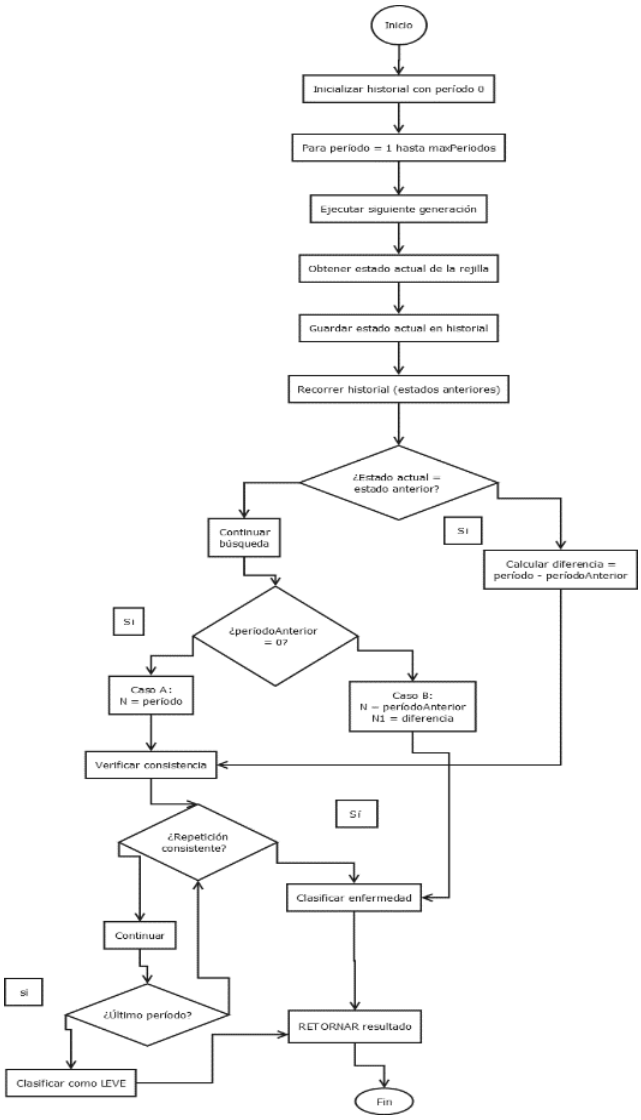

Figura 3. Diagrama de Flujo.

Fuente: Imagen propia.

- **Detección de Patrones**

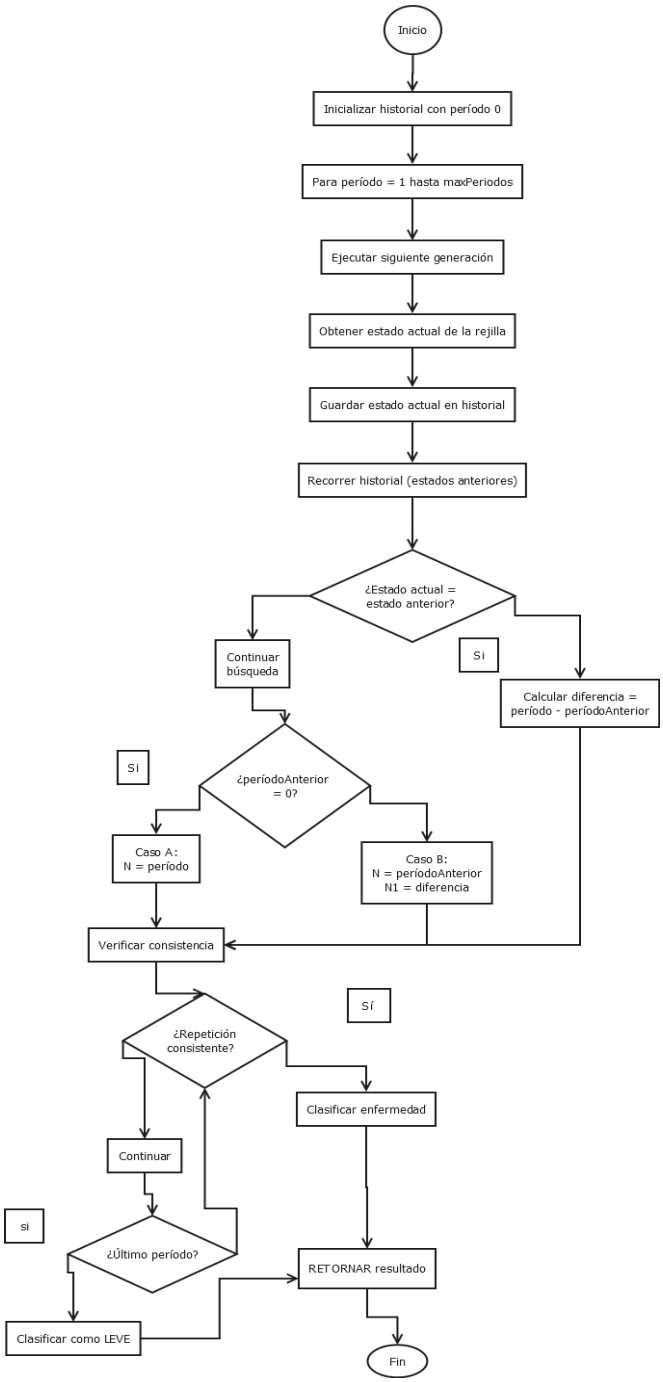

Figura 4. Diagrama de Flujo.

Fuente: Imagen propia.

- **Simulación**

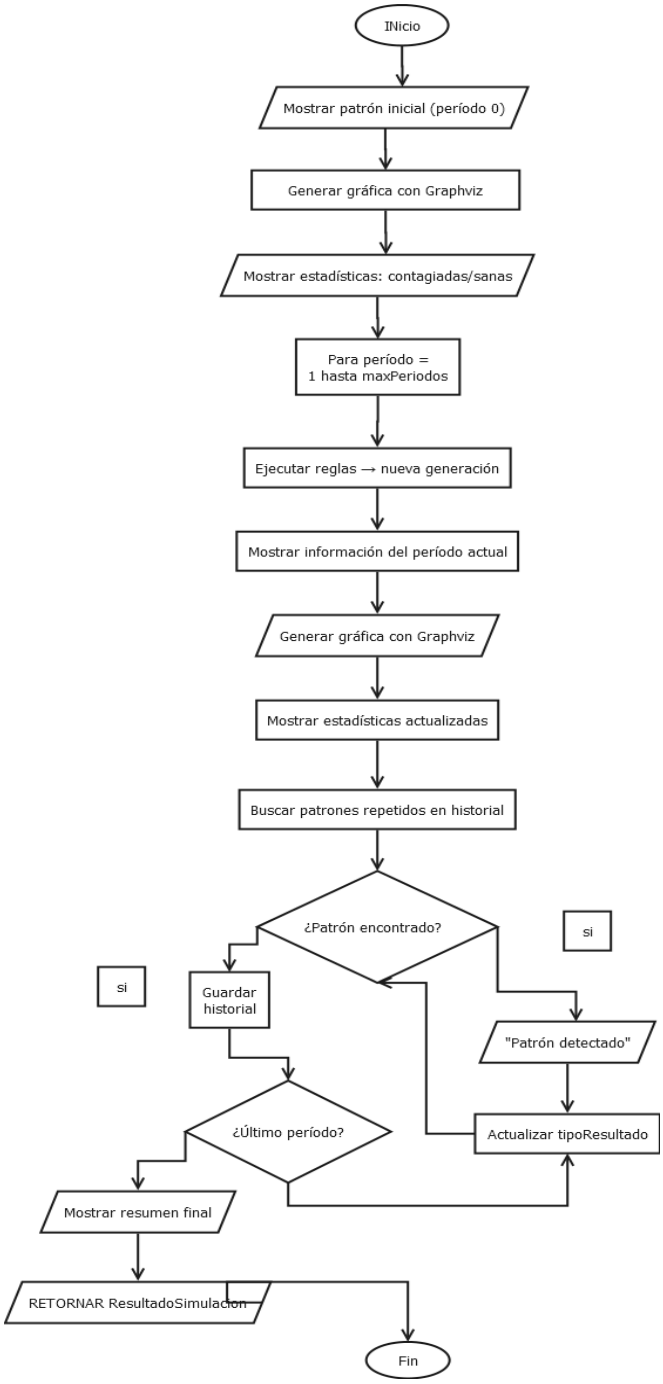

Figura 5. Diagrama de Flujo.
Fuente: Imagen propia.